

You said why do we get false sucess (venv) ───[Tue Sep 15 9:50:40 AM]

[root@Intel5]—[/home/remac/Desktop/youde...]

why do we get false sucess (venv) ───[Tue Sep 15 9:50:40 AM]

[root@Intel5]—[/home/remac/Desktop/youdemy/parrot-os-tweks/keyboard-touchpad]

└───[REMAC]# backupsysv2

Starting Backup script

Starting checks...

Drive match found... Checking Mount Point

Backup Drive Exists But Not Mounted. Attempting To Mount...

Mounted /dev/sda2 at /media/remac/LINUX_DATA

Drive Successfully Mounted.. Proceeding to backup Process

Backup Job completed Successfully

(venv) ───[Tue Sep 15 10:24:25 AM][root@Intel5]—[/home/remac/Desktop/youdemy/parrot-os-tweks/keyboard-touchpad]

└───[REMAC]#

even after landing on input output error from drive

(venv) ───[Tue Sep 15 10:25:14 AM][root@Intel5]—[/home/remac/Desktop/youdemy/parrot-os-tweks/keyboard-touchpad]

└───[REMAC]# cat `which backupsysv2`

#!/bin/bash

Author :- Remac

Created :- 04-07-2026

Color definitions

declare reset_f="\033[0m"

declare green_f="\033[32m"

declare yellow_f="\033[33m"

declare white_f="\033[37m"

declare red_f="\033[31m"

echo -e "\${green_f}Starting Backup script\${reset_f}"

Target Drive Identifiers

MATCH_LABEL="LINUX_DATA"

MATCH_UUID="b6975a27-d201-4cc1-8f96-e5668e5c5a3e"

MATCH_SIZE="233.8G"

MATCH_PARTUUID="210ddc2e-be80-ec41-bed0-3bd7745d9b82"

DRIVE_INFO=\$(lsblk -no LABEL,UUID,SIZE,PARTUUID,MOUNTPOINT /dev/disk/by-uuid/"\$MATCH_UUID" 2>/dev/null)

read -r CHECK_LABEL CHECK_UUID CHECK_SIZE CHECK_PARTUUID MOUNTPOINT <<< "\$DRIVE_INFO"

startup_checks(){

printf "\${yellow_f}Starting checks...\n\${reset_f}"

if [["\$MATCH_LABEL" == "\$CHECK_LABEL" && "\$MATCH_UUID" == "\$CHECK_UUID" && "\$MATCH_SIZE" == "\$CHECK_SIZE" && "\$MATCH_PARTUUID" == "\$CHECK_PARTUUID"]]; then

printf "\${yellow_f}Drive match found... Checking Mount Point\n\${reset_f}"

if [[-n "\$MOUNTPOINT"]]; then

printf "\${white_f}Mount Point Exists (\${MOUNTPOINT}).. Proceeding with backup\n\${reset_f}"

else

printf "\${red_f}Backup Drive Exists But Not Mounted. Attempting To Mount...\n\${reset_f}"

sudo -u remac udiskscctl mount --block-device /dev/disk/by-uuid/"\$MATCH_UUID"

if [[\$? -eq 0]]; then

printf "\${white_f}Drive Successfully Mounted.. Proceeding to backup Process\n\${reset_f}"

else

printf "\${red_f}Error Mounting Drive.. Aborting\n\${reset_f}"

exit 1

fi

```
fi
else
printf "${red_f}Drive not found or signature mismatch, Quitting...\n${reset_f}"
exit 1
fi
}

startup_checks

# Run rsync in xterm, save exit code, wait for Enter, and propagate exit code
xterm -font 10x20 -geometry 124x27 -T 'Full System BackUp Script' -e bash -c '
rsync -avPh --inplace --info=progress2 \
--exclude={"/dev/*","/proc/*","/sys/*","/tmp/*","/run/*","/media/*","/lost+found"} \
--exclude="/amazon_linux.tar" \
--exclude="/amazon_linux" \
/ /media/remac/LINUX_DATA/
RSYNC_EXIT=$?
echo ""
if [ $RSYNC_EXIT -eq 0 ]; then
echo -e "\033[32m[+] Backup completed successfully.\033[0m"
elif [ $RSYNC_EXIT -eq 130 ]; then
echo -e "\033[31m[-] Backup aborted by user (Ctrl+C).\033[0m"
else
echo -e "\033[31m[-] Backup failed with exit code: $RSYNC_EXIT\033[0m"
fi

read -p "Press [ENTER] to close window and finish script..."
exit $RSYNC_EXIT
'

# Capture xterm's exit status (which is now identical to rsync's status)
JOB_STATUS=$?
if [[ $JOB_STATUS -eq 0 ]]; then
printf "${white_f}Backup Job completed Successfully\n${reset_f}"
elif [[ $JOB_STATUS -eq 130 ]]; then
printf "${red_f}Backup Job interrupted by user (Exit 130)\n${reset_f}"
else
printf "${red_f}Something went wrong (Exit Code: %d)\n${reset_f}" "$JOB_STATUS"
fi

(venv) ┌───[Tue Sep 15 10:25:14 AM][root@Intel5]—[/home/remac/Desktop/youdemy/parrot-os-tweks/keyboard-
touchpad]
└───[REMAC]#
```

YOU

You said why do we get false sucess (venv) ┌───[Tue Sep 15 9:50:40 AM]
[root@Intel5]—[/home/remac/Desktop/youde...]

Your script reports false success because of how `xterm -e bash -c '...'` handles subshell exit codes and string evaluation.

Here are the primary flaws causing this behavior:

Key Technical Issues

Unescaped `$RSYNC_EXIT` Inside Double Quotes

In the main script, your string inside `xterm -e bash -c '...'` uses double quotes for `bash -c "..."`. Because of variable expansion, `RSYNC_EXIT=$?` and `$RSYNC_EXIT` are being evaluated by the parent script before `xterm` even runs, rather than inside the new `xterm` window.

`xterm` Exit Status Masking

When `xterm` finishes, its standard return code often reflects whether `xterm` itself launched and exited cleanly, rather than passing through the exit status of the child shell inside it.

Missing `set -e` or Signal Propagation

If `rsync` encounters non-fatal Input/Output errors on individual unreadable files, it exits with status 23 or 24 (Partial transfer / vanished files) rather than 0. If variable evaluation breaks inside the nested string, `$JOB_STATUS` defaults back to 0.

How to Fix It

Instead of passing a complex inline multiline command to `xterm`, write the status directly to a temporary file, or use proper escaping and structure. The cleanest way is to avoid embedding unescaped subshell commands inside `xterm`.

Here is the updated, reliable version of your script:

Bash

```
#!/bin/bash
```

```
# Author :- Remac
```

```
# Updated :- Fix false success on rsync failure
```

```
declare reset_f="\033[0m"
```

```
declare green_f="\033[32m"
```

```
declare yellow_f="\033[33m"
```

```
declare white_f="\033[37m"
```

```
declare red_f="\033[31m"
```

```
echo -e "${green_f}Starting Backup script${reset_f}"
```

```
MATCH_LABEL="LINUX_DATA"
```

```
MATCH_UUID="b6975a27-d201-4cc1-8f96-e5668e5c5a3e"
```

```
MATCH_SIZE="233.8G"
```

```
MATCH_PARTUUID="210ddc2e-be80-ec41-bed0-3bd7745d9b82"
```

```
DRIVE_INFO=$(lsblk -no LABEL,UUID,SIZE,PARTUUID,MOUNTPOINT /dev/disk/by-uuid/"$MATCH_UUID" 2>/dev/null)
```

```
read -r CHECK_LABEL CHECK_UUID CHECK_SIZE CHECK_PARTUUID MOUNTPOINT <<< "$DRIVE_INFO"
```

```
startup_checks(){
```

```
printf "${yellow_f}Starting checks...\n${reset_f}"
```

```
if [[ "$MATCH_LABEL" == "$CHECK_LABEL" && "$MATCH_UUID" == "$CHECK_UUID" && "$MATCH_SIZE" == "$CHECK_SIZE" && "$MATCH_PARTUUID" == "$CHECK_PARTUUID" ]]; then
```

```
printf "${yellow_f}Drive match found... Checking Mount Point\n${reset_f}"
```

```
if [[ -n "$MOUNTPOINT" ]]; then
```

```
printf "${white_f}Mount Point Exists ($MOUNTPOINT).. Proceeding with backup\n${reset_f}"
```

```
else
```

```
printf "${red_f}Backup Drive Exists But Not Mounted. Attempting To Mount...\n${reset_f}"
```

```
sudo -u remac udiskctl mount --block-device /dev/disk/by-uuid/"$MATCH_UUID"
```

```
if [[ $? -eq 0 ]]; then
```

```
printf "${white_f}Drive Successfully Mounted.. Proceeding to backup Process\n${reset_f}"
```

```
else
```

```
printf "${red_f}Error Mounting Drive.. Aborting\n${reset_f}"
```

```
exit 1
```

```
fi
```

```

fi
else
printf "${red_f}Drive not found or signature mismatch, Quitting...\n${reset_f}"
exit 1
fi
}

startup_checks
STATUS_FILE=$(mktemp)

# Run rsync inside xterm and write exit code to temporary file
xterm -font 10x20 -geometry 124x27 -T 'Full System BackUp Script' -e bash -c '
rsync -avPh --inplace --info=progress2 \
--exclude={"/dev/*","/proc/*","/sys/*","/tmp/*","/run/*","/media/*","/lost+found"} \
--exclude="/amazon_linux.tar" \
--exclude="/amazon_linux" \
/ /media/remac/LINUX_DATA/
RSYNC_EXIT=$?
echo "$RSYNC_EXIT" > ""$STATUS_FILE""
echo ""
if [ $RSYNC_EXIT -eq 0 ]; then
echo -e "\033[32m[+] Backup completed successfully.\033[0m"
elif [ $RSYNC_EXIT -eq 130 ]; then
echo -e "\033[31m[-] Backup aborted by user (Ctrl+C).\033[0m"
else
echo -e "\033[31m[-] Backup failed with exit code: $RSYNC_EXIT\033[0m"
fi

read -p "Press [ENTER] to close window and finish script..."
'

# Read actual exit status from file
JOB_STATUS=$(cat "$STATUS_FILE" 2>/dev/null || echo 1)
rm -f "$STATUS_FILE"

# How to verify: Check if status properly reflects rsync error code
if [[ "$JOB_STATUS" -eq 0 ]]; then
printf "${white_f}Backup Job completed Successfully\n${reset_f}"
elif [[ "$JOB_STATUS" -eq 130 ]]; then
printf "${red_f}Backup Job interrupted by user (Exit 130)\n${reset_f}"
else
printf "${red_f}Something went wrong (Exit Code: %d)\n${reset_f}" "$JOB_STATUS"
fi

```

Verification

Run the updated script when an I/O error or missing drive occurs. The script will correctly output Something went wrong (Exit Code: <code_number>) instead of returning Backup Job completed Successfully.